GPU architecture: Revisiting the SIMT execution model

January 2020

Caroline Collange
Inria Rennes – Bretagne Atlantique
https://team.inria.fr/pacap/members/collange/
caroline.collange@inria.fr

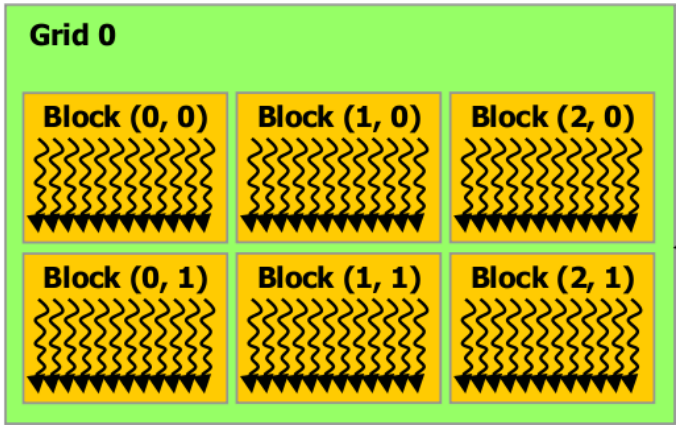
Outline

- Running SPMD software on SIMD hardware
  - Context: software and hardware
  - The control flow divergence problem
- Stack-based control flow tracking
  - Stacks
  - Counters
- Path-based control flow tracking
  - The idea: use PCs
  - Implementation: path list
  - Applications

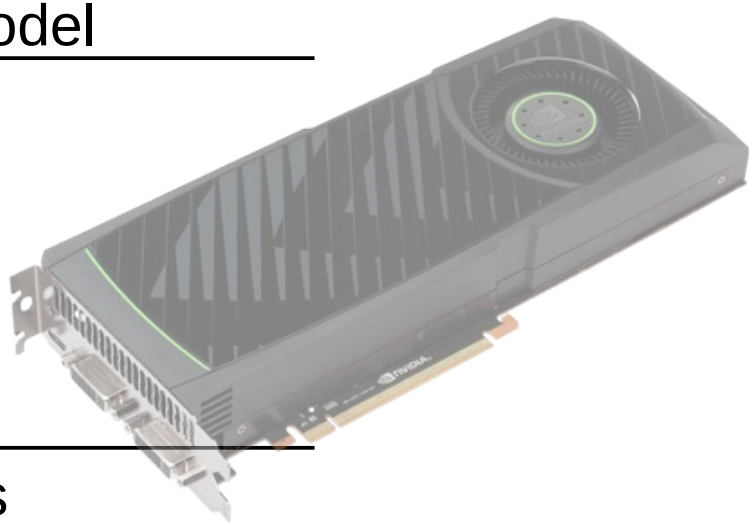
GPU microarchitecture: where it fits

Software

```
__global__ void scale(float a, float * X)
{
    unsigned int tid;
    tid = blockIdx.x * blockDim.x
          + threadIdx.x;
    X[tid] = a * X[tid];
}
```

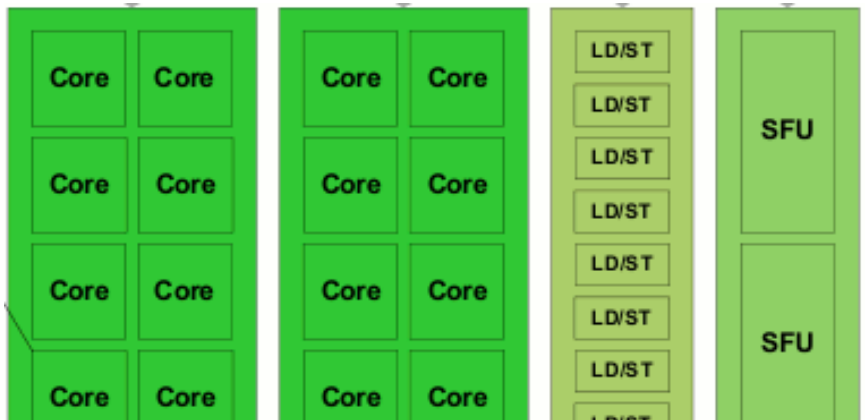


Architecture: **multi-thread** programming model



SIMT microarchitecture

Hardware datapaths: **SIMD** execution units



Hardware

Warps

- Threads are grouped into warps of fixed size

Warp 0				Warp 1			
T0	T1	T2	T3	T4	T5	T6	T7

An early SIMT architecture. *Musée Gallo-Romain de St-Romain-en-Gal, Vienne*

Control flow: uniform or divergent

- Control is ***uniform*** when all threads in the warp follow the same path

- Control is ***divergent*** when different threads follow different paths

```
x = 0;
// Uniform condition
if(a[x] > 17) {
      x = 1;
}
// Divergent condition
if(tid < 2) {
      x = 2;
}
```

Outcome per thread:

	Warp			
	T0	T1	T2	T3
Uniform condition	1	1	1	1
Divergent condition	1	1	0	0

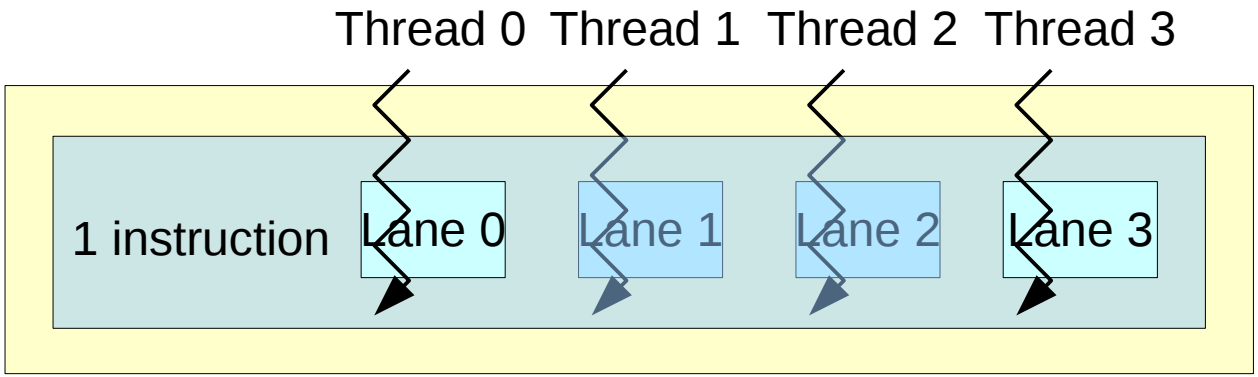
Computer architect view

- SIMD execution inside a warp
  - One SIMD lane per thread
  - All SIMD lanes see the same instruction

- Control-flow differentiation using **execution mask**
  - All instructions controlled with 1 bit per lane
    1 → perform instruction
    0 → do nothing

Running independent threads in SIMD

- How to keep threads synchronized?
  - Challenge: divergent control flow
- Rules of the game
  - One thread per SIMD lane
  - **Same instruction** on all lanes
  - Lanes can be individually disabled with **execution mask**
- Which instruction?
- How to compute execution mask?




```
x = 0;
// Uniform condition
if(tid > 17) {
      x = 1;
}
// Divergent conditions
if(tid < 2) {
      if(tid == 0) {
            x = 2;
      }
      else {
            x = 3;
      }
}
```

Example: if

Execution mask inside if statement = if condition

```
x = 0;
// Uniform condition
if(a[x] > 17) {
      x = 1;
}
```

	Warp			
	T0	T1	T2	T3
`if` condition: a[x] > 17?	1	1	1	1
Execution mask:	1	1	1	1

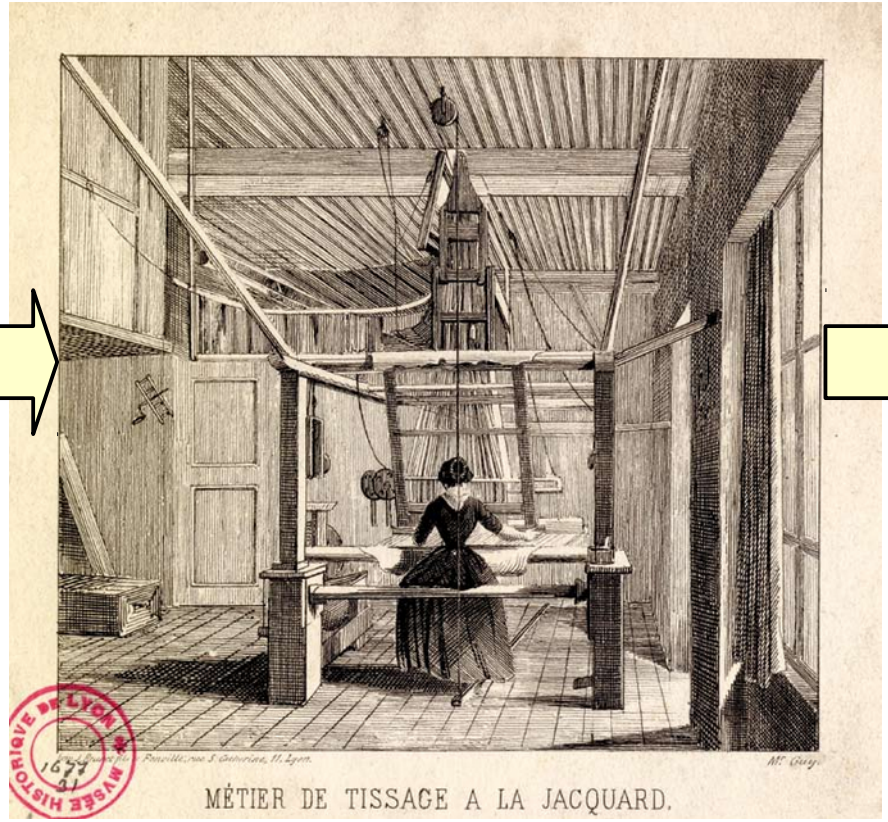
```
// Divergent condition
if(tid < 2) {
      x = 2;
}
```

`if` condition: tid < 2?	1	1	0	0
Execution mask:	1	1	0	0

State of the art in 1804

- The Jacquard loom is a GPU

Shaders



Framebuffer

- Multiple warp threads with per-thread conditional execution
  - Execution mask given by punched cards
  - Supports 600 to 800 parallel threads

Conditionals that no thread executes

- Do not waste energy fetching instructions not executed
  - Skip instructions when execution mask is all-zeroes

```
x = 0;
// Uniform condition
if(a[x] > 17) {
        x = 1;
}
else {
        x = 0;
}
// Divergent condition
if(tid < 2) {
        x = 2;
}
```

	Warp			
	T0	T1	T2	T3
`if` condition: a[x] > 17?	1	1	1	1
Execution mask:	1	1	1	1
Execution mask:	0	0	0	0

Jump to end-if

`if` condition: tid < 2?	1	1	0	0
Execution mask:	1	1	0	0

- Uniform branches are just usual scalar branches

What about loops?

- Keep looping until all threads exit
  - Mask out threads that have exited the loop

```
i = 0;
while(i < tid) {
    i++;
}
print(i);
```

Execution trace:

		Warp			
		T0	T1	T2	T3
`i = 0;`	i =?	0	0	0	0
	i < tid?	0	1	1	1
`i++;`	i =?	0	1	1	1
	i < tid?	0	0	1	1
`i++;`	i =?	0	1	2	2
	i < tid?	0	0	0	1
`i++;`	i =?	0	1	2	3
	i < tid?	0	0	0	0

No active thread left → restore mask and exit loop

`print(i);`	i =?	0	1	2	3

Time

What about nested control flow?

```
x = 0;
// Uniform condition
if(tid > 17) {
      x = 1;
}
// Divergent conditions
if(tid < 2) {
      if(tid == 0) {
          x = 2;
      }
      else {
          x = 3;
      }
}
```

- We need a generic solution!

Outline

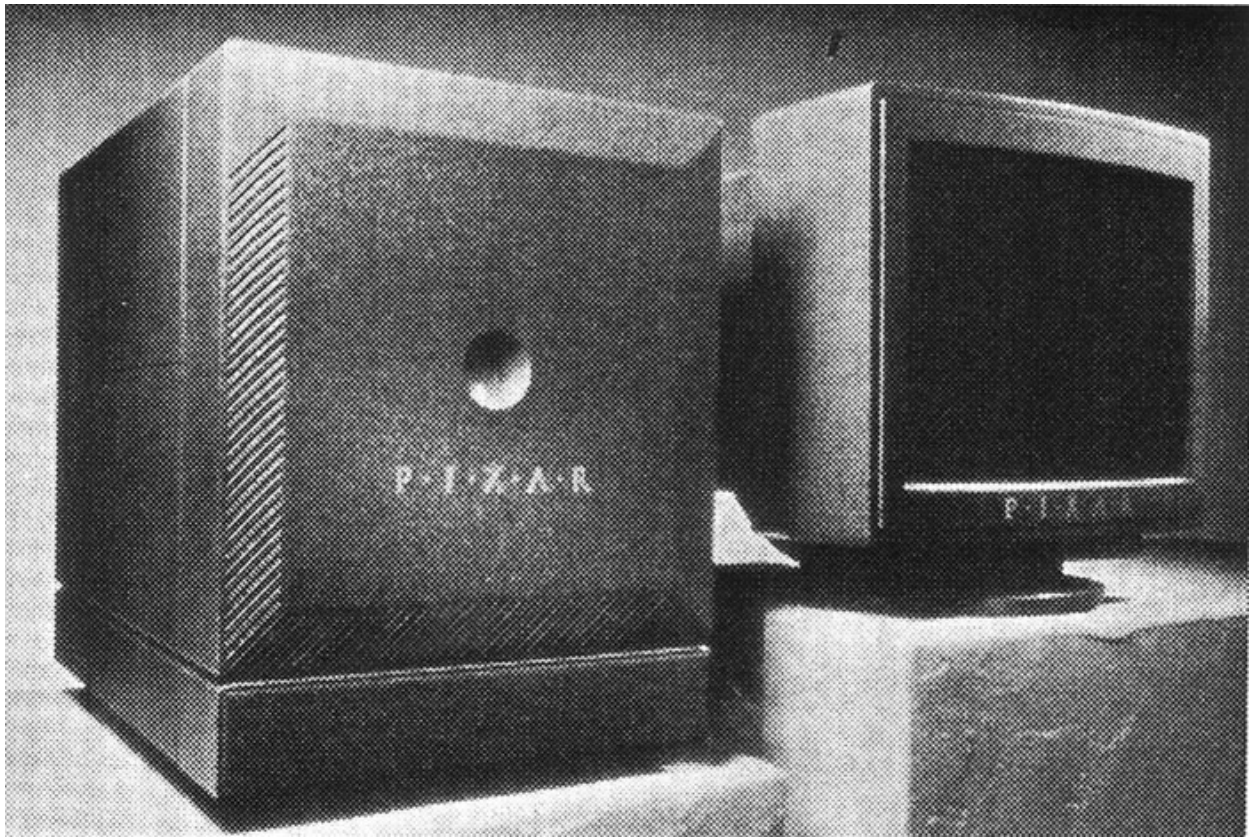
- Running SPMD software on SIMD hardware
  - Context: software and hardware
  - The control flow divergence problem
- Stack-based control flow tracking
  - Stacks
  - Counters
- Path-based control flow tracking
  - The idea: use PCs
  - Implementation: path list
  - Applications

Quizz

- What do *Star Wars* and GPUs have in common?

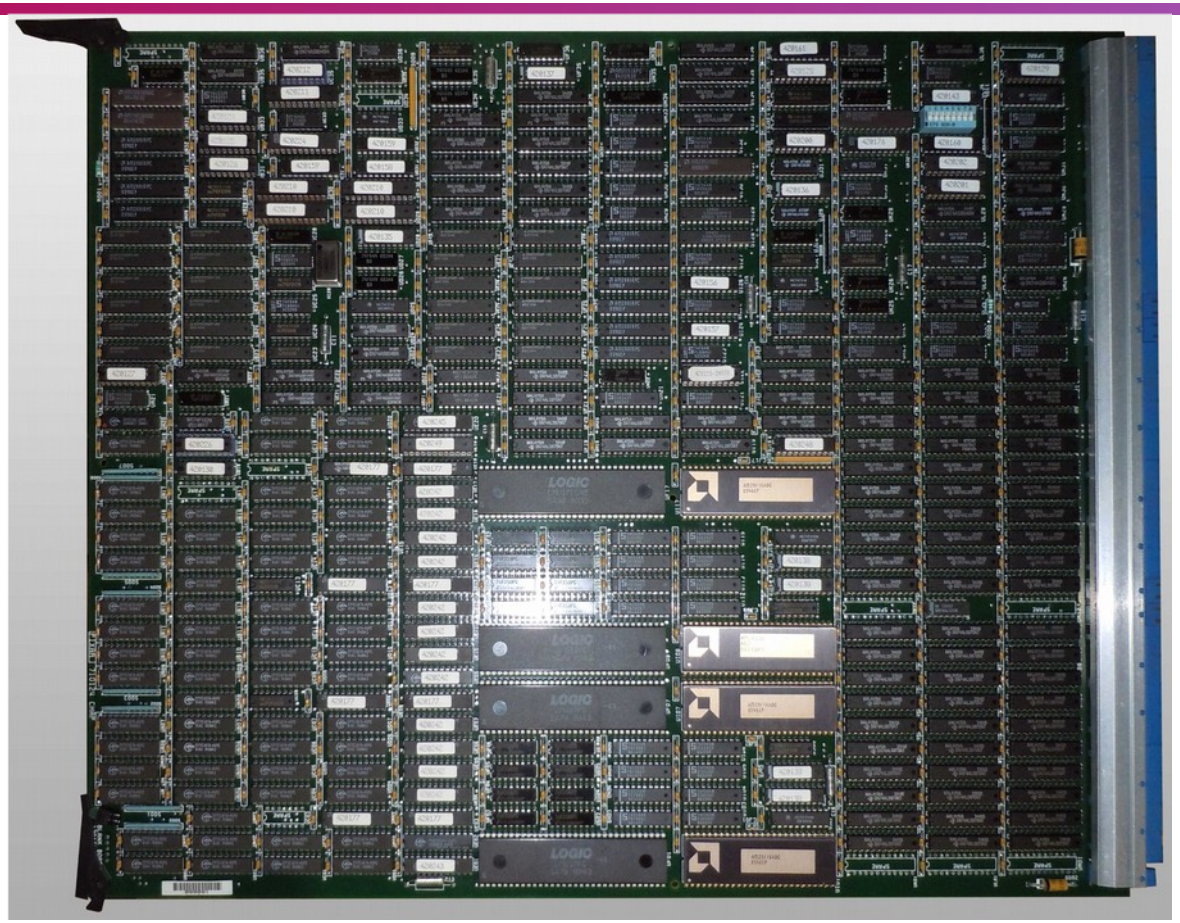
Answer: Pixar!

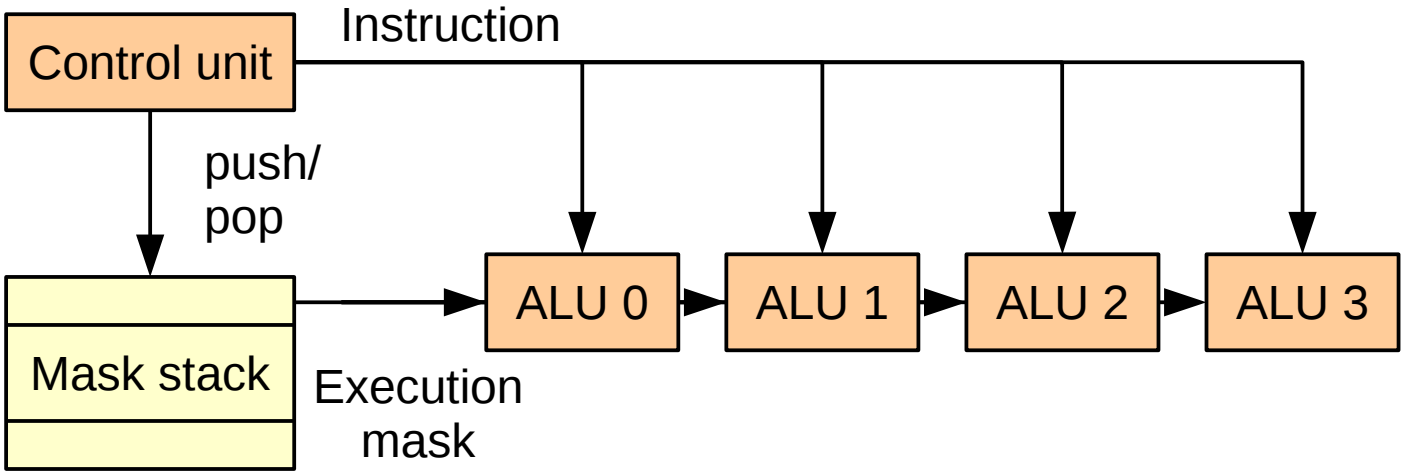
- In the early 1980's, the *Computer Division* of *Lucasfilm* was designing custom hardware for computer graphics
- Acquired by Steve Jobs in 1986 and became *Pixar*
- Their core product: the *Pixar Image Computer*



- This early GPU handles nested divergent control flow!

Pixar Image Computer: architecture overview





The mask stack of the Pixar Image Computer

Code

```
x = 0;
// Uniform condition
if(tid > 17) {
        x = 1;
}
// Divergent conditions
if(tid < 2) {
    if(tid == 0) {
        x = 2;
    }
    else {
        x = 3;
    }
}
```

skip

push

push

pop

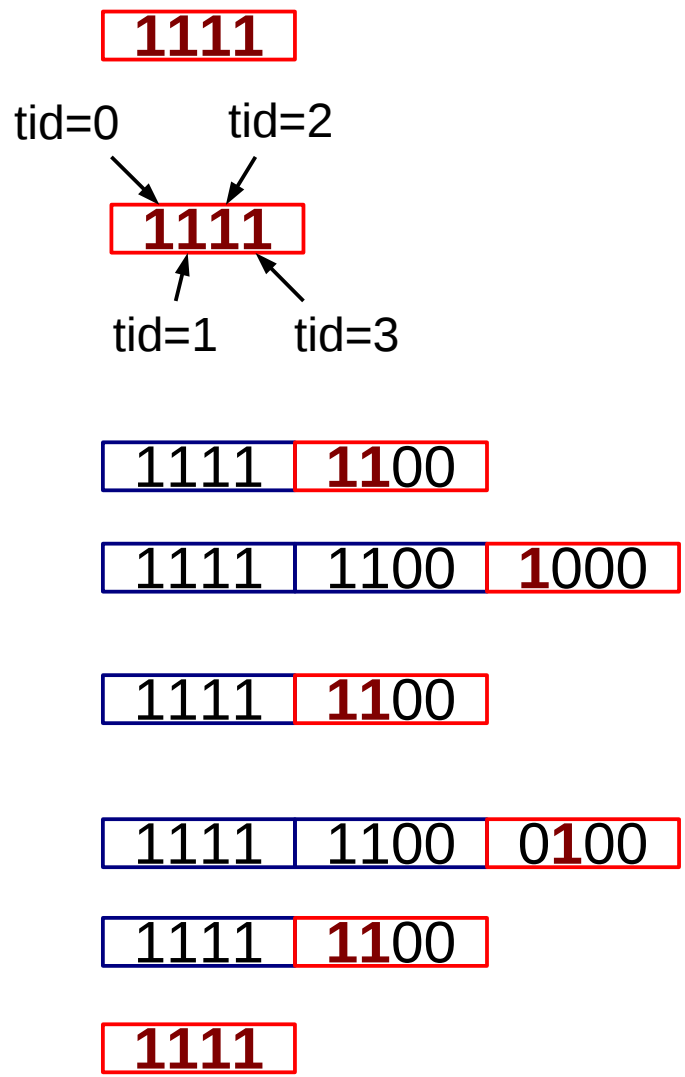
push

pop

pop

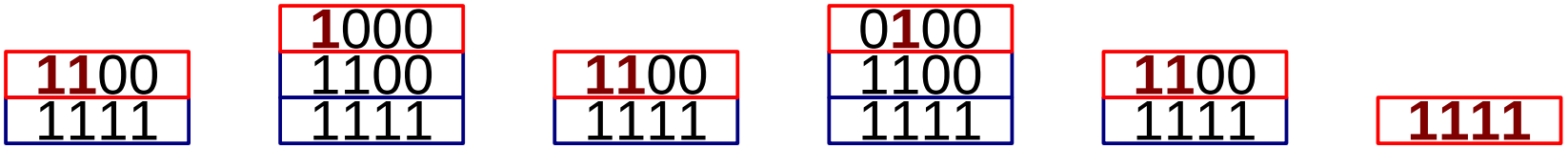
Mask Stack

1 activity bit / thread



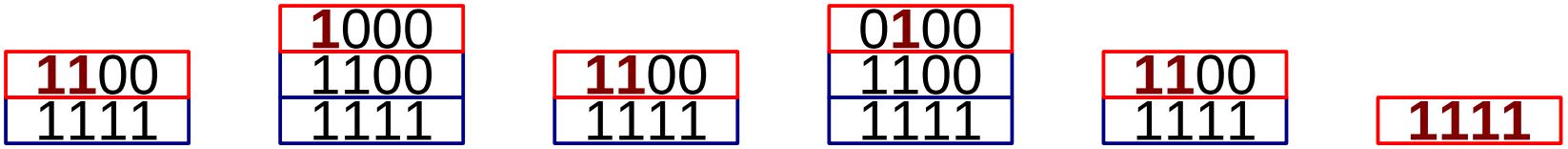
A. Levinthal and T. Porter. *Chap - a SIMD graphics processor*. SIGGRAPH'84, 1984.

Observation: stack content is a histogram



- On structured control flow: columns of 1s
  - A thread active at level *n* is active at all levels *i*<*n*
  - Conversely: no “zombie” thread gets revived at level *i*>*n* if inactive at *n*

Observation: stack content is a histogram



- On structured control flow: columns of 1s
  - A thread active at level *n* is active at all levels *i*<*n*
  - Conversely: no “zombie” thread gets revived at level *i*>*n* if inactive at *n*

- The height of each column of 1s is enough
  - Alternative implementation: maintain an activity counter for each thread

R. Keryell and N. Paris. *Activity counter : New optimization for the dynamic scheduling of SIMD control flow.* ICPP ’93, 1993.

With activity counters

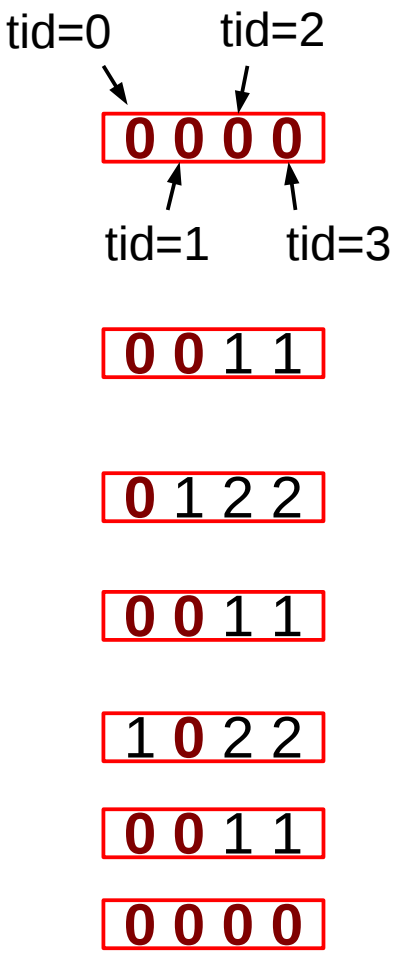
Code

```
x = 0;
// Uniform condition
if(tid > 17) {
        x = 1;
}
// Divergent conditions
if(tid < 2) {
        if(tid == 0) {
                x = 2;
        }
        else {
                x = 3;
        }
}
```

skip
inc
inc
dec
inc
dec
dec

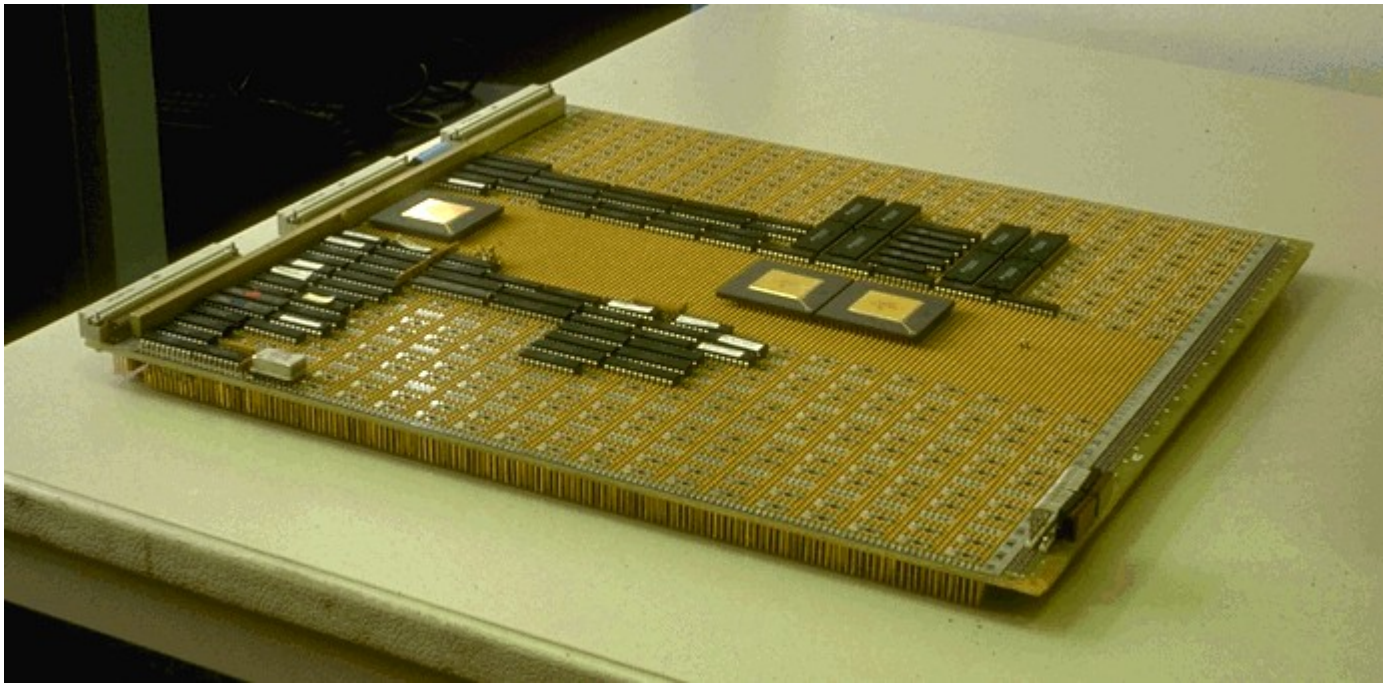
Counters

1 (in)activity counter / thread



Activity counters in use

- In an SIMD prototype from École des Mines de Paris in 1993



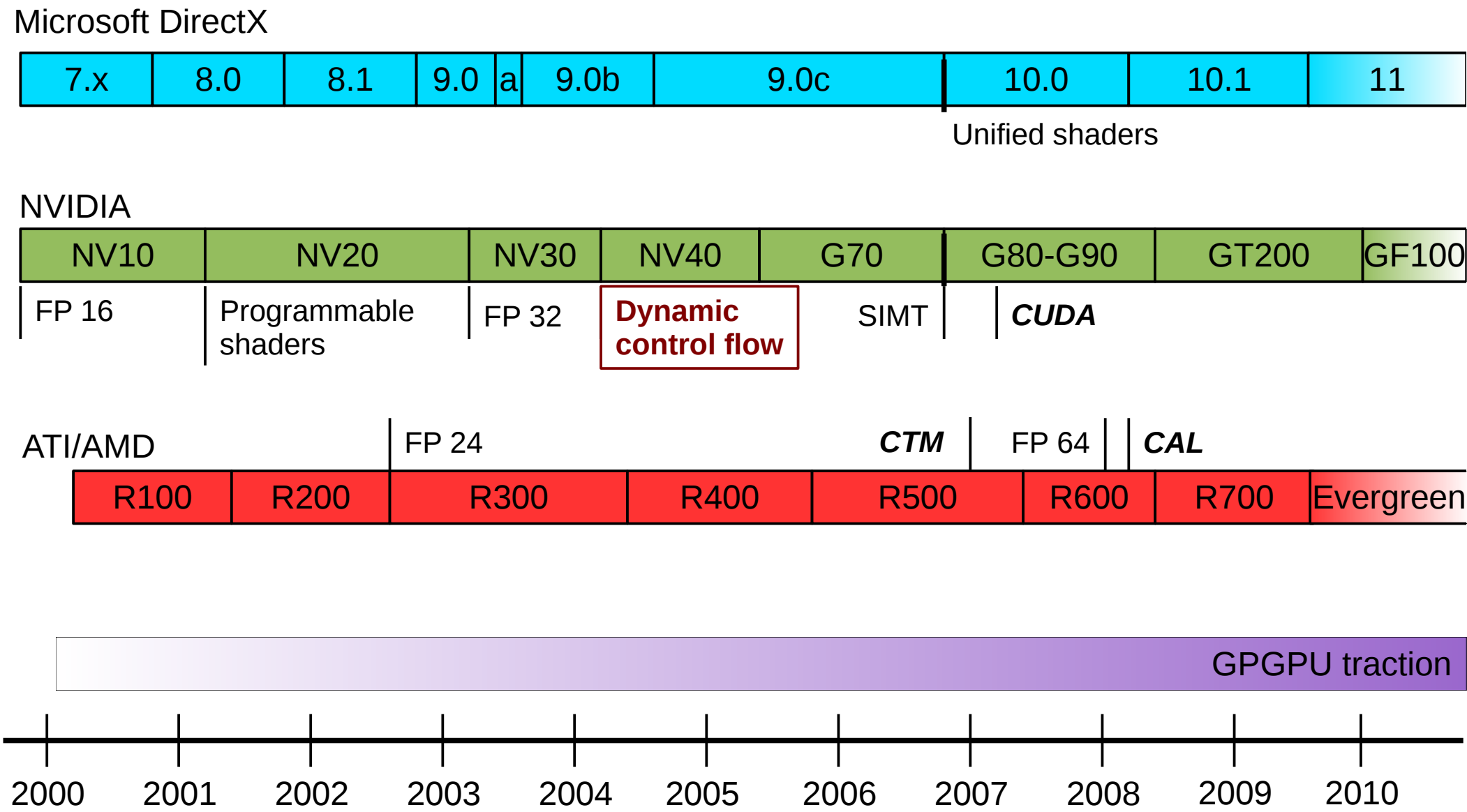
Credits: Ronan Keryell

- In Intel integrated graphics since 2004

Outline

- Running SPMD software on SIMD hardware
  - Context: software and hardware
  - The control flow divergence problem
- Stack-based control flow tracking
  - Stacks, counters
  - A compiler perspective
- Path-based control flow tracking
  - The idea: use PCs
  - Implementation: path list
  - Applications
- Software approaches
  - Use cases and principle
  - Scalarization
- Research directions

Back to ancient history



Early days of programmable shaders

It is 21st century!

- Graphics cards now look and sound like hair dryers

The infamous GeForce FX 5800 Ultra

- Graphics shaders are programmed in assembly-like language
  - Direct3D shader assembly, OpenGL ARB Vertex/Fragment Program...
  - Control-flow: `if`, `else`, `endif`, `while`, `break`… are assembly instructions

- Graphics driver performs a straightforward translation to GPU-specific machine language

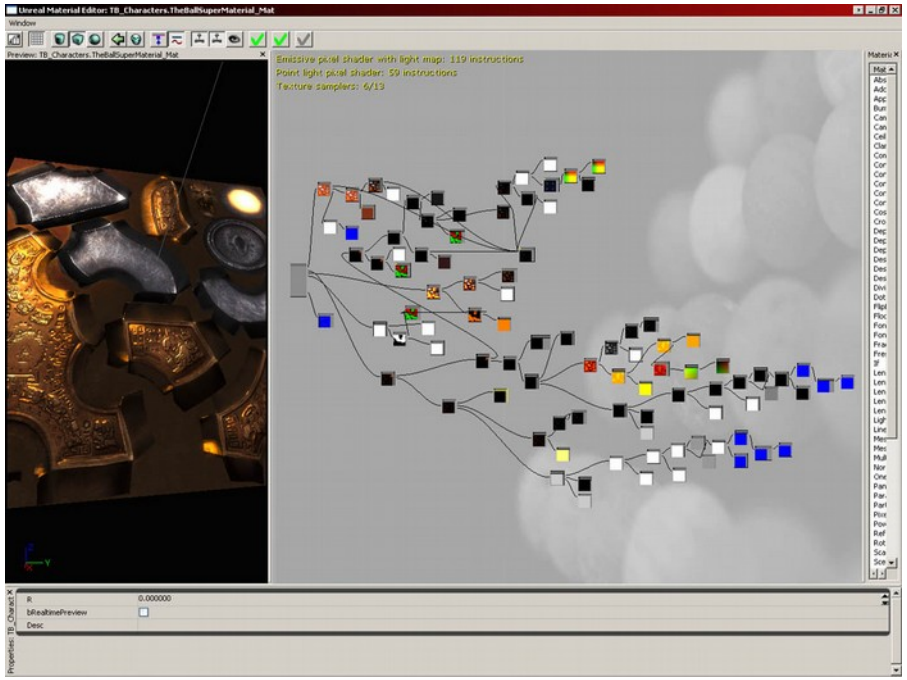
Goto considered harmful?

MIPS	NVIDIA Tesla (2007)	NVIDIA Fermi (2010)	Intel GMA Gen4 (2006)	Intel GMA SB (2011)	AMD R500 (2005)	AMD R600 (2007)	AMD Cayman (2011)
`j`	`bar`	`bar`	`jmpi`	`jmpi`	`jump`	`push`	`push`
`jal`	`bra`	`bpt`	`if`	`if`	`loop`	`push_else`	`push_else`
`jr`	`brk`	`bra`	`iff`	`else`	`endloop`	`pop`	`pop`
`syscall`	`brkpt`	`brk`	`else`	`endif`	`rep`	`loop_start`	`push_wqm`
	`cal`	`brx`	`endif`	`case`	`endrep`	`loop_start_no_al`	`pop_wqm`
	`cont`	`cal`	`do`	`while`	`breakloop`	`loop_start_dx10`	`else_wqm`
	`kil`	`cont`	`while`	`break`	`breakrep`	`loop_end`	`jump_any`
	`pbk`	`exit`	`break`	`cont`	`continue`	`loop_continue`	`reactivate`
	`pret`	`jcal`	`cont`	`halt`		`loop_break`	`reactivate_wqm`
	`ret`	`jmx`	`halt`	`call`		`jump`	`loop_start`
	`ssy`	`kil`	`msave`	`return`		`else`	`loop_start_no_al`
	`trap`	`pbk`	`mrest`	`fork`		`call`	`loop_start_dx10`
	`.s`	`pret`	`push`			`call_fs`	`loop_end`
		`ret`	`pop`			`return`	`loop_continue`
		`ssy`				`return_fs`	`loop_break`
		`.s`				`alu`	`jump`
						`alu_push_before`	`else`
						`alu_pop_after`	`call`
						`alu_pop2_after`	`call_fs`
						`alu_continue`	`return`
						`alu_break`	`return_fs`
						`alu_else_after`	`alu`
							`alu_push_before`
							`alu_pop_after`
							`alu_pop2_after`
							`alu_continue`
							`alu_break`
							`alu_else_after`

Control instructions in some CPU and GPU instruction sets

- GPUs: instruction set expresses control flow structure
  - Where should we stop?

Next: compilers for GPU code



- High-level shader languages
  - C-like: HLSL, GLSL, Cg
  - Then visual languages (UDK)
- General-purpose languages
  - CUDA, OpenCL
  - Then directive-based: OpenACC, OpenMP 4
  - Python (Numba)…
- Incorporate function calls, switch-case, && and ||…
- Demands a compiler infrastructure
  - A Just-In-Time compiler in graphics drivers

A typical GPU compiler

- First: turns all structured control flow into gotos, generates intermediate representation (IR)
  - e.g. Nvidia PTX, Khronos SPIR, llvm IR
- Then: performs various compiler optimizations on IR
- Finally: reconstructs structured control flow back from gotos to emit machine code
  - Not necessarily the same as the original source!

Issues of stack-based implementations

If GPU threads are actual threads, they can synchronize?

- e.g. using semaphores, mutexes, condition variables…
- Problem: SIMT-induced livelock

```
while(!acquire(lock)) {}
…
release(lock)
```

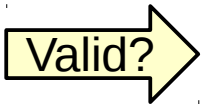
Example: critical section
Thread 0 acquires the lock,
keeps looping with other threads of the warp waiting for the lock.
Infinite loop, lock never released.

- Stack-based SIMT divergence control can cause starvation!

Issues of stack-based implementations

- Are all control flow optimizations valid in SIMT?

```
f();
```




```
if(c)
  f();
else
  f();
```

- What about context switches?
  - e.g. migrate one single thread of a warp
  - Challenging to do with a stack

- Truly general-purpose computing demands more flexible techniques

Outline

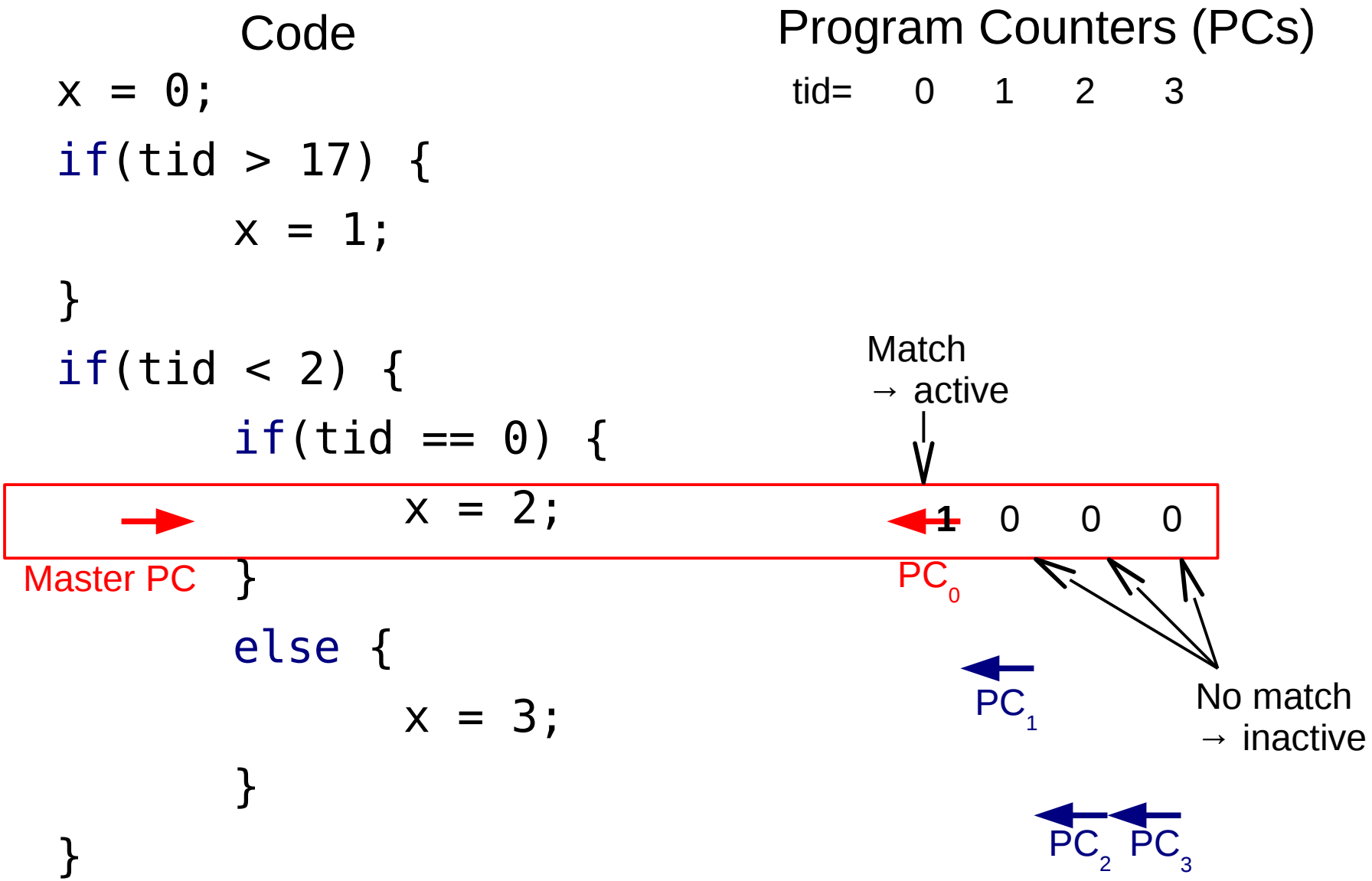
- Running SPMD software on SIMD hardware
  - Context: software and hardware
  - The control flow divergence problem
- Stack-based control flow tracking
  - Stacks, counters
  - A compiler perspective
- Path-based control flow tracking
  - The idea: use PCs
  - Implementation: path list
  - Applications

With 1 PC / thread

Code

```
x = 0;
if(tid > 17) {
        x = 1;
}
if(tid < 2) {
        if(tid == 0) {
                x = 2;
        }
        else {
                x = 3;
        }
}
```

Program Counters (PCs)



Mask stacks vs. per-thread PCs

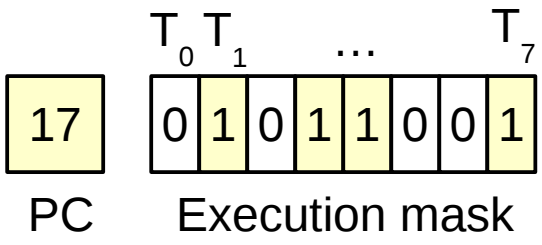
- Before: stack, counters
  - O(n), **O(log n)** memory
    n = nesting depth
  - **1 R/W port** to memory
  - **Exceptions**: stack overflow, underflow
- Vector semantics
  - Structured control flow only
  - Specific instruction sets

- After: multiple PCs
  - **O(1)** memory
  - **No shared state**
  - Allows thread **suspension, restart, migration**
- Multi-thread semantics
  - Traditional languages, compilers
  - **Traditional instruction sets**
- **Can be mixed with MIMD**

- **Straightforward implementation is more expensive**

Path-based control flow tracking

- A **path** is characterized by a PC and execution mask



- The mask encodes the **set of threads** that have this PC

$\{ T_1, T_3, T_4, T_7 \}$ have PC 17

A list of paths represents a vector of PCs

PC_0	PC_1	PC_2	PC_3	PC_4	PC_5	PC_6	PC_7
12	17	3	17	17	3	3	17

Per-thread PCs

	T_0	T_1						T_7
CPC_1	3	0	0	1	0	0	1	1	0
CPC_2	12	1	0	0	0	0	0	0	0
CPC_3	17	0	1	0	1	1	0	0	1

List of paths

- Worst case: 1 path per thread
  - Path list size is bounded
- PC vector and path list are **equivalent**
  - You can switch freely between MIMD thinking and SIMD thinking!

Pipeline overview

- Select an active path

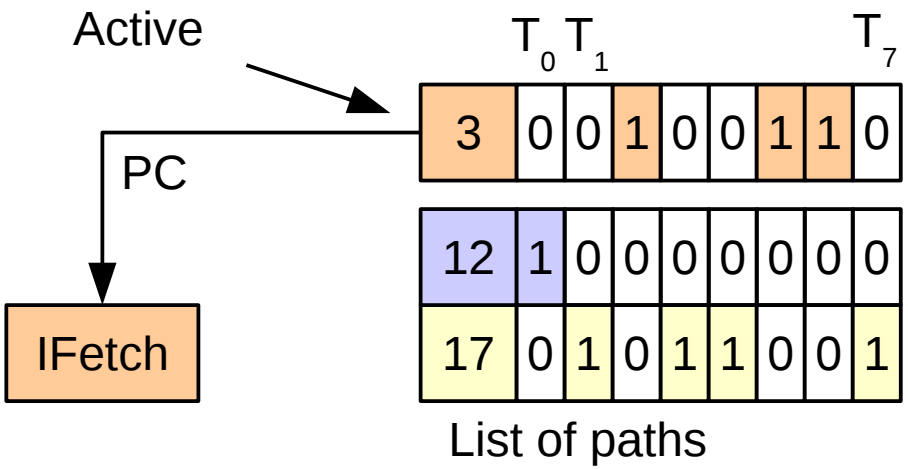
Active

	T_0	T_1						T_7
3	0	0	1	0	0	1	1	0
12	1	0	0	0	0	0	0	0
17	0	1	0	1	1	0	0	1

List of paths

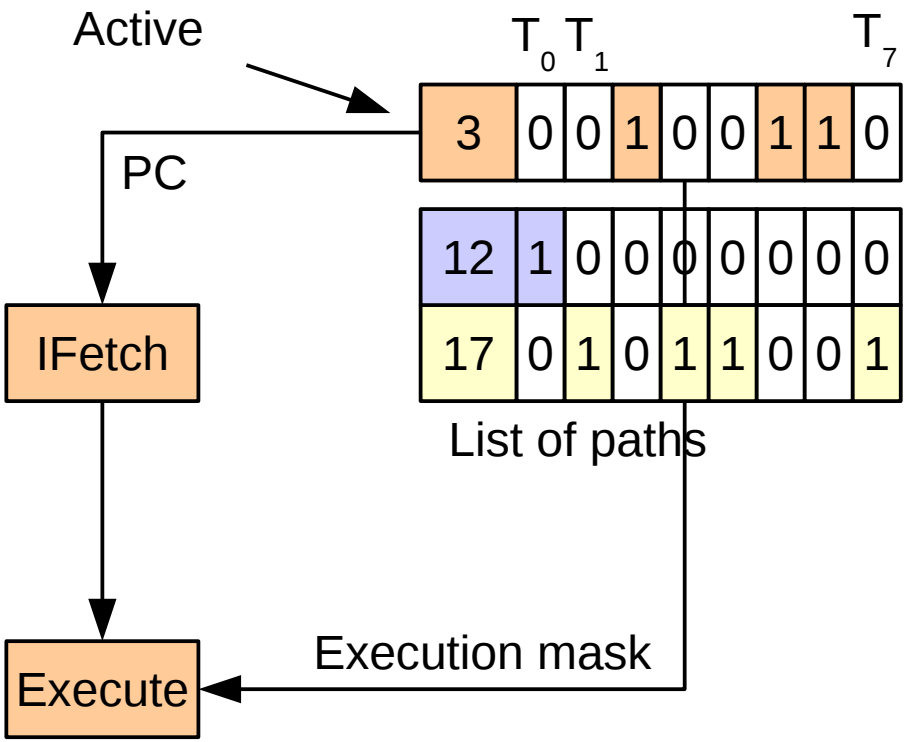
Pipeline overview

- Select an active path
- Fetch instruction at PC of active path



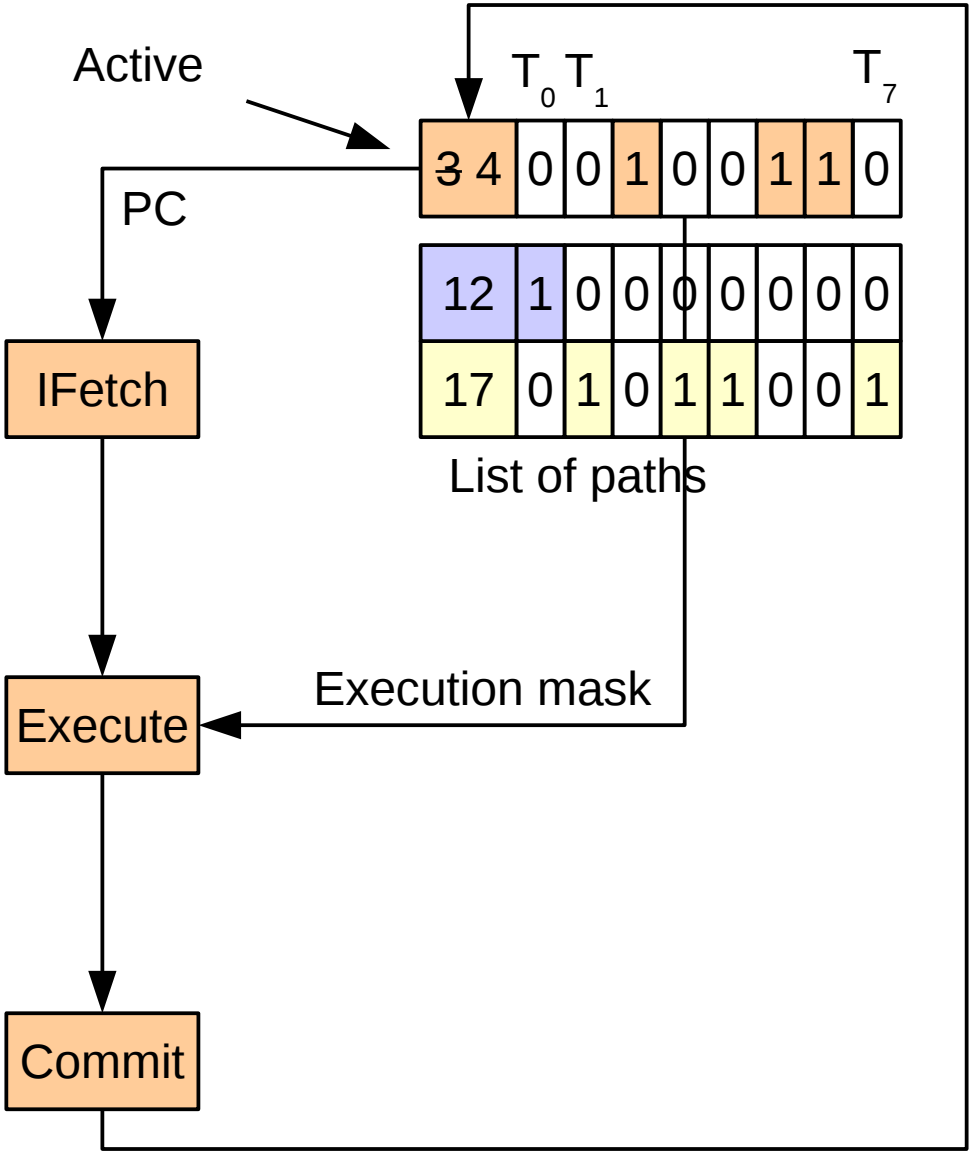
Pipeline overview

- Select an active path
- Fetch instruction at PC of active path
- Execute with execution mask of active path



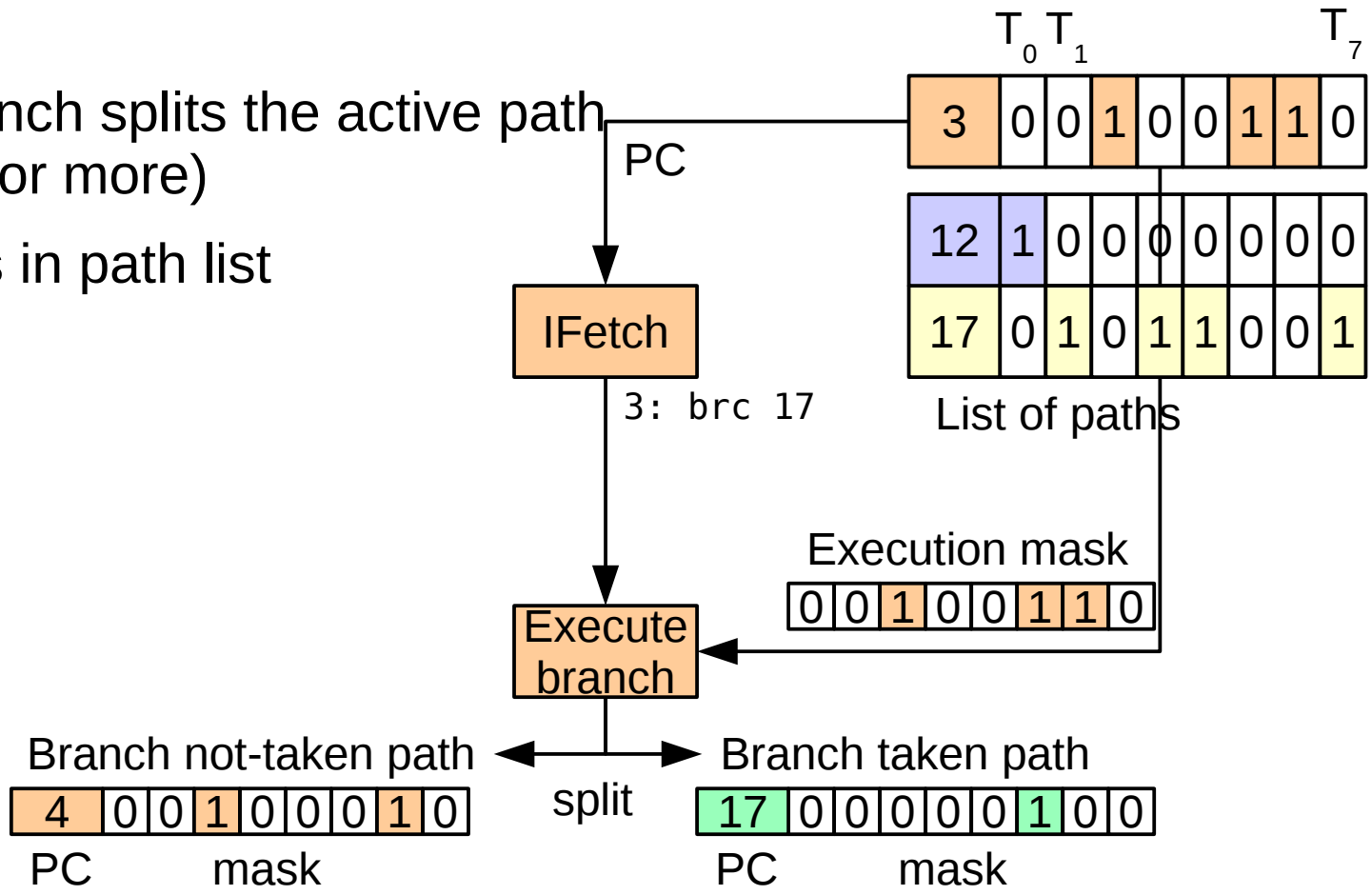
Pipeline overview

- Select an active path
- Fetch instruction at PC of active path
- Execute with execution mask of active path
- For uniform instruction: update PC of active path



Divergent branch is path insertion

- A divergent branch splits the active path into two paths (or more)
- Insert the paths in path list



New path table

4	0	0	1	0	0	0	1	0
12	1	0	0	0	0	0	0	0
17	0	1	0	1	1	0	0	1
17	0	0	0	0	0	1	0	0

Convergence is path fusion

- When two paths have the same PC, we can merge them
  - New set of threads is the union of former sets
  - New execution mask is bitwise OR of former masks

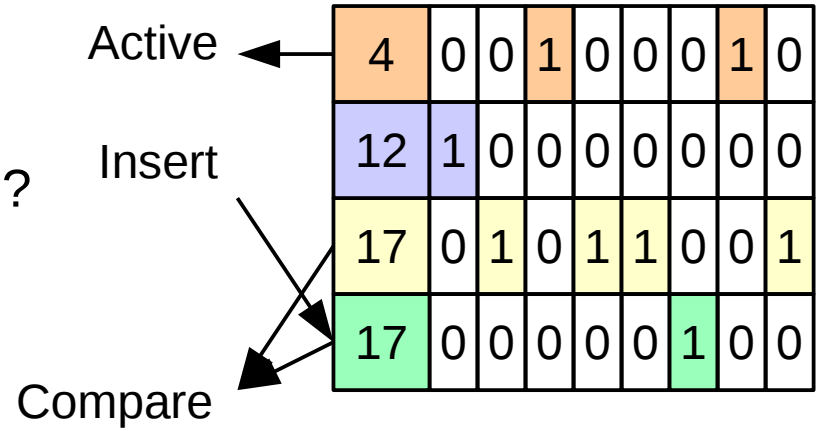
4	0	0	1	0	0	0	1	0
12	1	0	0	0	0	0	0	0
17	0	1	0	1	1	0	0	1
17	0	0	0	0	0	1	0	0

17	0	1	0	1	1	1	0	1

4	0	0	1	0	0	0	1	0
12	1	0	0	0	0	0	0	0
17	0	1	0	1	1	1	0	1

Path scheduling is graph traversal

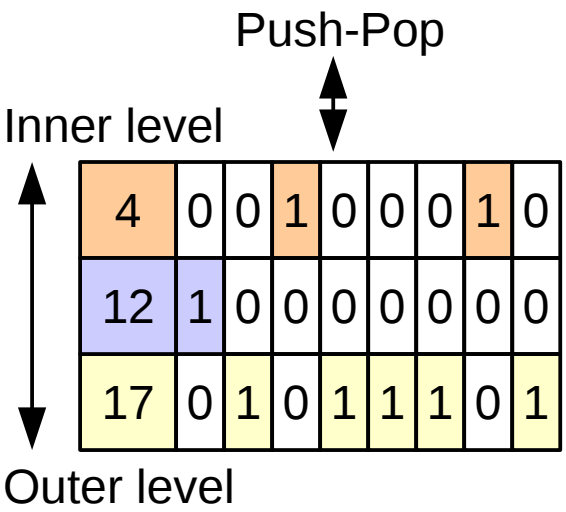
- Degrees of freedom
  - Which path is the active path?
  - At which place are new paths inserted?
  - When and where do we check for convergence?



- Different answers yield different policies

Depth-first graph traversal

- Remember graph algorithm theory
  - Depth-first graph traversal using a stack worklist
- Path list as a stack = depth-first traversal of the control-flow graph
  - Most deeply nested levels first



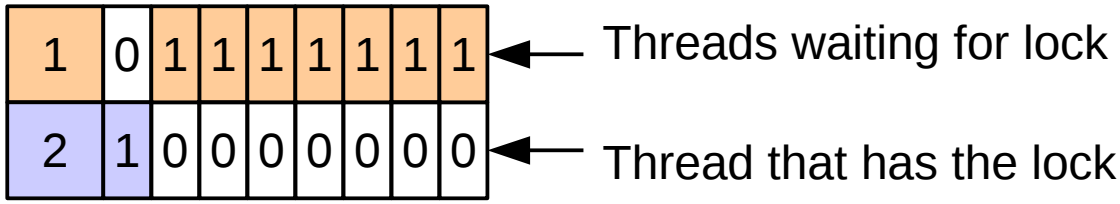

```
x = 0;
// Uniform condition
if(tid > 17) {
      x = 1;
}
// Divergent conditions
if(tid < 2) {
      if(tid == 0) {
            x = 2;
      }
      else {
            x = 3;
      }
}
```

Question: is this the same as Pixar-style mask stack? Why?

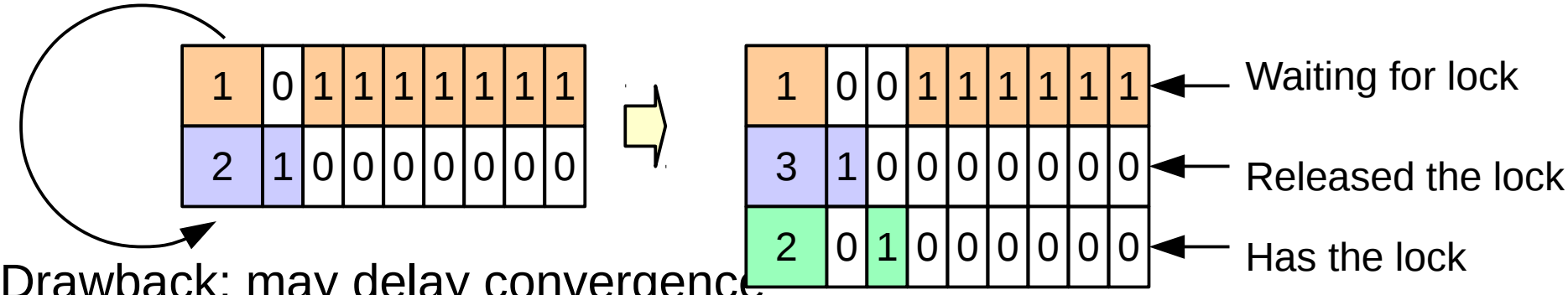
Breadth-first graph traversal

- Goal: guarantee forward progress to avoid SIMT-induced livelocks

```
  while(!acquire(lock)) {
1:
  }
2:…
  release(lock)
3:
```



- Path list as a queue: follow paths in round-robin



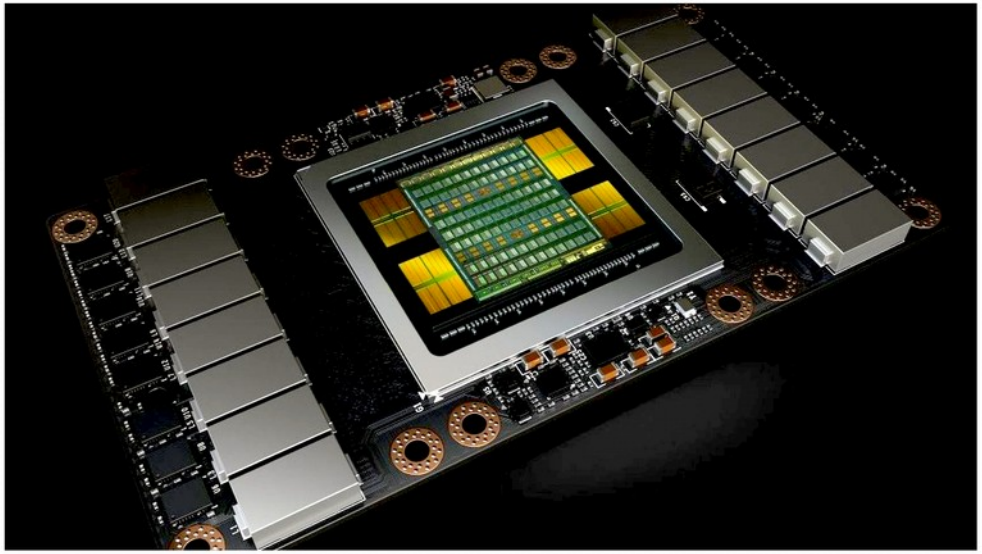
- Drawback: may delay convergence

A. ElTantawy and T. Aamodt. MIMD Synchronization on SIMT Architectures. Micro 2016

Example: Nvidia Volta (2017)

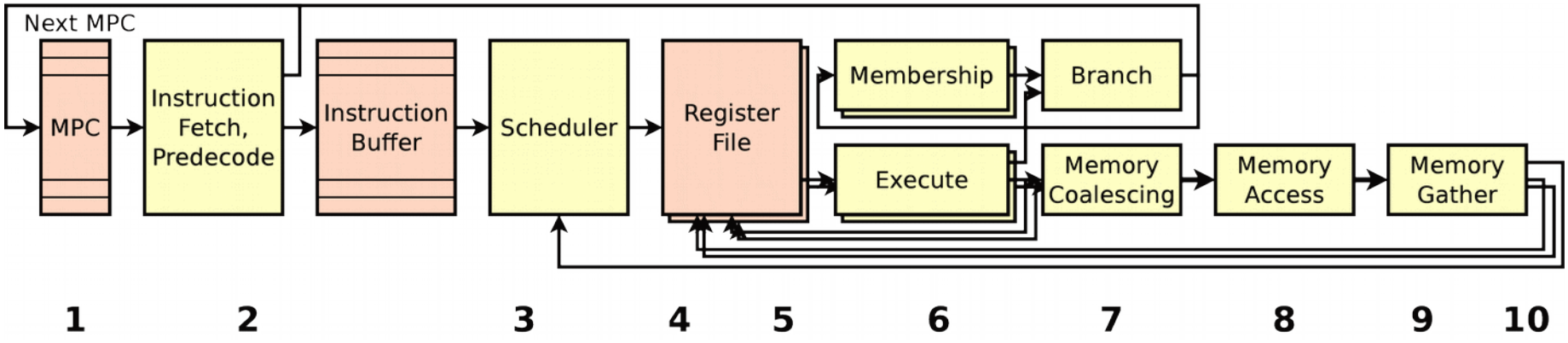
Supports independent thread scheduling inside a warp

- Threads can synchronize with each other inside a warp
- Diverged threads can run barriers (as long as all threads eventually reach a barrier)



Advertisement: Simty, a SIMT CPU

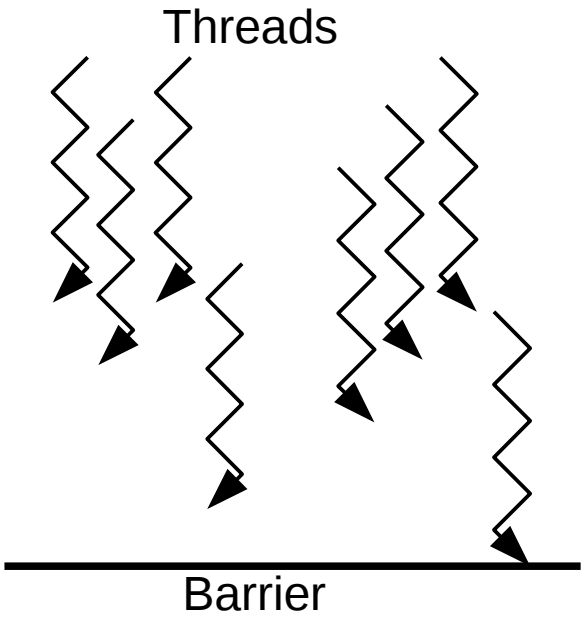
- Proof of concept for priority-based SIMT
  - Written in synthesizable VHDL
  - Runs the RISC-V instruction set (RV32I)
  - Fully parametrizable warp size, warp count
  - 10-stage pipeline



https://team.inria.fr/pacap/simty/

Programmer's view

- Programming model
  - SPMD: Single program, multiple data
  - One *kernel* code, many *threads*
  - Unspecified execution order between explicit synchronization barriers



- Languages
  - Graphics shaders : HLSL, Cg, GLSL
  - GPGPU : C for CUDA, OpenCL

```
For n threads:
    X[tid] ← a * X[tid]
```

Kernel

Types of control flow

- **Structured** control flow:
  single-entry, single exit
  properly nested
  - Conditionals: `if-then-else`
  - Single-entry, single-exit loops: `while, do-while, for...`
  - Function call-return…
- **Unstructured** control flow
  - `break, continue`
  - `&& ||` short-circuit evaluation
  - Exceptions
  - Coroutines
  - `goto, comefrom`
  - Code that is hard to indent!